

myClinic

Software Design Specification

24.02.2026

Student Id	Name Surname
150122054	Eren Emre Aycibin
150122008	Nursena Kıyga
150122010	Hüseyin Aksoy
150122037	Sinem Yenibalcı

Prepared for
CSE3044 Software Engineering Term Project

Table of Contents

Table of Contents	1
1. Introduction.....	4
1.1 Purpose.....	4
1.2 Statement of Scope	4
1.3 Software Context	5
1.4 Major Constraints.....	5
1.5 Definitions.....	5
1.6 Acronyms and Abbreviations	5
1.7 References	6
2. Design Considerations	7
2.1 Design Assumptions and Dependencies	7
2.2 General Constraints.....	7
2.3 System Environment.....	8
2.4 Development Methods	8
3. Architectural and Component-Level Design	9
3.1 System Structure	9
3.1.1 Architecture Diagram.....	9
3.2 C1: Authentication & Authorization Module	10
3.2.1 Processing Narrative (PSPEC).....	10
3.2.2 Interface Description.....	10
3.2.3 Processing Detail	10
3.2.4 Dynamic Behavior: Interaction Diagram.....	11
3.3 C2: Appointment & Dynamic Pricing Module.....	11
3.3.1 Processing Narrative (PSPEC).....	11
3.3.2 Interface Description.....	11
3.3.3 Processing Detail	12
3.3.4 Dynamic Behavior: Interaction Diagram.....	12
3.4 C3: AI Integration Module.....	13
3.4.1 Processing Narrative (PSPEC).....	13
3.4.2 Interface Description.....	13
3.4.3 Processing Detail	14
3.4.4 Dynamic Behavior: Interaction Diagram.....	14
3.5 C4: Medical Records & Prescription Module.....	15

3.5.1 Processing Narrative (PSPEC).....	15
3.5.2 Interface Description.....	15
3.5.3 Processing Detail	16
3.5.4 Dynamic Behavior: Interaction Diagram.....	16
3.6 C5: Secure Messaging Module	16
3.6.1 Processing Narrative (PSPEC).....	17
3.6.2 Interface Description.....	17
3.6.3 Processing Detail	17
3.6.4 Dynamic Behavior: Interaction Diagram.....	17
3.7 C6: Analytics & Leave Management Module	18
3.7.1 Processing Narrative (PSPEC).....	18
3.7.2 Interface Description.....	18
3.7.3 Processing Detail	19
3.7.4 Dynamic Behavior: Interaction Diagrams	19
4. Data Model.....	21
5. Restrictions, Limitations, and Constraints	22
6. Conclusion	23
Appendix A: Decision Log	24
Decision 1: MVC (Layered) Architecture over Microservices.....	24
Decision 2: Isolating AI in a Single Integration Module (C3)	24
Decision 3: PostgreSQL Row-Level Locking for Concurrent Bookings	25
Decision 4: JWT for Stateless Authentication (over Django Sessions).....	25
Appendix B: AI Tools Usage Declaration	26

1. Introduction

This section is carried over from the Software Requirements Specification Document. It provides background information about the myClinic project necessary to contextualize the design decisions described throughout this document.

1.1 Purpose

The purpose of this Software Design Specification (DSD) is to describe the architectural and component-level design of the myClinic web application. This document translates the requirements defined in the SRS into a concrete design blueprint that guides the implementation and testing phases.

The overall software objectives are:

- Provide a scalable, role-based web platform for clinic management.
- Integrate AI capabilities for dynamic pricing and medical test analysis.
- Ensure data security, compliance with KVKK/GDPR, and high availability.
- Support modular design allowing independent component updates.

1.2 Statement of Scope

myClinic is a web-based clinic management system for small-to-medium clinics. It supports three user roles, namely Patient, Doctor, and Manager, and automates clinical workflows including appointment scheduling, digital prescriptions, leave management, secure messaging, analytics, and AI-assisted medical analysis.

Functional requirements are ranked by development priority:

Requirement	Priority	Justification
FR-1: Appointment Scheduling & AI Pricing	Essential	Core clinic operation, primary user interaction
FR-3: Doctor Leave Management	Essential	Required for schedule integrity
FR-6: Digital Prescription Generation	Essential	Core medical workflow
FR-4: Secure Patient-Doctor Messaging	Desirable	Enhances UX; encryption adds complexity
FR-5: Manager Analytics Dashboard	Desirable	Valuable but not critical to core operation
FR-2: AI-Assisted Medical Test Analysis	Future	Depends on external AI API stability and prompt quality

1.3 Software Context

myClinic positions itself as a modernization tool for small and medium-sized private clinics currently relying on paper-based or fragmented digital processes. It is inspired conceptually by large national health systems (e.g., Turkey's E-Nabız) but scoped for a single-clinic deployment.

The system operates as a standalone web application deployed on a cloud server, communicating with external services (AI API, SMTP provider) via RESTful endpoints. It stores all persistent data in a relational SQL database. This context makes the design decisions around modularity, security, and graceful degradation particularly important.

1.4 Major Constraints

- Backend must be Python-based (Django or FastAPI framework).
- Database must be a relational SQL system (PostgreSQL preferred).
- All data handling must comply with KVKK/GDPR data privacy regulations.
- AI-assisted features depend on a third-party LLM API (e.g., OpenAI); unavailability must be handled gracefully.
- System must be deployable as a web application accessible via modern browsers without any client-side installation.
- The project must be implementable and testable within one semester.

1.5 Definitions

Term	Definition
RBAC	Role-Based Access Control, an authorization model that restricts system access based on user roles.
ORM	Object-Relational Mapper, a tool that abstracts database interactions through object-oriented code (e.g., Django ORM).
PSPEC	Processing Specification: a narrative description of a component's responsibilities.
Dynamic Pricing	An algorithm that adjusts appointment fees in real-time based on demand and availability.
Graceful Degradation	System continues core operation with reduced functionality when a dependent service fails.
JWT	JSON Web Token, a compact and URL-safe token used for stateless authentication between client and server.

1.6 Acronyms and Abbreviations

Acronym	Meaning
SRS	Software Requirements Specification
DSD	Design Specification Document
UI	User Interface
API	Application Programming Interface
REST	Representational State Transfer
DB	Database
AI	Artificial Intelligence
LLM	Large Language Model
MVC	Model-View-Controller
SMTP	Simple Mail Transfer Protocol
TLS	Transport Layer Security
ACID	Atomicity, Consistency, Isolation, Durability
KVKK	Kişisel Verilerin Korunması Kanunu (Turkish Personal Data Protection Law)
GDPR	General Data Protection Regulation

1.7 References

- IEEE Std 1016-2009, IEEE Standard for Information Technology: Systems Design, Software Design Descriptions.
- CSE3044 Software Engineering Course Materials, Marmara University, Spring 2025-2026.
- myClinic Software Requirements Specification, v1.0, March 2026.
- Django REST Framework Documentation: <https://www.django-rest-framework.org>
- PostgreSQL 16 Official Documentation: <https://www.postgresql.org/docs/>

2. Design Considerations

This section describes the issues, assumptions, and constraints that influenced the design of myClinic before a detailed design solution was formulated.

2.1 Design Assumptions and Dependencies

The following assumptions and dependencies were identified during the design phase:

- Users access the system via modern web browsers (Chrome, Firefox, Safari, and Edge, latest two major versions) on desktop or mobile devices.
- All users have a stable internet connection; the system is not designed for offline operation.
- A third-party LLM API (e.g., OpenAI API) is available and accessible from the server for AI features (dynamic pricing, test analysis). API key management and cost are outside the system's responsibility.
- A third-party SMTP provider (e.g., SendGrid or AWS SES) is configured and available for email notifications.
- The deployment server runs a Linux-based OS (Ubuntu 22.04+) with Python 3.10+ and PostgreSQL 15+ installed.
- Clinic staff will accurately enter and maintain patient and doctor data in the system.
- AI outputs are treated as advisory; they do not trigger automated medical actions.

2.2 General Constraints

The following global constraints have a significant impact on the design:

- **Hardware/Software Environment:** The application is deployed on a cloud VM (e.g., AWS EC2, DigitalOcean Droplet). The frontend communicates with the backend exclusively over HTTPS (TLS 1.2+).
- **Standards Compliance:** All data at rest containing PII or medical records must be encrypted with AES-256. Passwords must be hashed using bcrypt or Argon2.
- **Interoperability:** The system communicates with external services (AI API, SMTP) exclusively via RESTful/JSON interfaces.
- **Memory and Capacity:** The system must support at least 500 concurrent users without performance degradation (per NFR).
- **Performance:** 95% of page loads/DB queries must complete within 2.0 seconds; AI API interactions within 5.0 seconds.
- **Security:** Strict RBAC must be enforced at every API endpoint. Sessions expire after 15 minutes of inactivity.
- **Verification and Validation:** Unit test coverage must be maintained above 80%. All core business logic components must have corresponding unit tests.

2.3 System Environment

Layer	Technology / Tool	Purpose
Backend Framework	Python / Django (with Django REST Framework)	API logic, ORM, RBAC middleware
Database	PostgreSQL 15+	Persistent relational data storage
Frontend	React.js (or Django Templates for MVP)	Role-based dashboards and UI
AI Integration	OpenAI API (GPT-4o or similar)	Dynamic pricing & test analysis
Email Service	SendGrid / AWS SES (SMTP)	Automated notifications
Authentication	JWT (via djangorestframework-simplejwt)	Stateless session management
Task Scheduler	Celery + Redis	Background jobs (email reminders, cron)
Version Control	Git / GitHub	Source control and team collaboration
Deployment	Linux VM (Ubuntu 22.04), Nginx, Gunicorn	Web server and application hosting
Testing	pytest / Django TestCase	Unit and integration testing

2.4 Development Methods

The system is designed following a layered MVC (Model-View-Controller) architectural pattern, implemented through Django's MVT (Model-View-Template) structure and its REST Framework. This choice was made after considering two alternative approaches:

- **MVC / Layered Architecture (Selected):** Provides clear separation between data models, business logic, and presentation. Django's built-in ORM and DRF middleware enforce this separation. This is well-suited to the team's familiarity with Python/Django, and the modular structure supports independent updates to components (e.g., swapping the AI module without touching appointment logic).
- **Microservices Architecture (Rejected):** While offering greater scalability, microservices introduce significant DevOps overhead (service discovery, inter-service communication, container orchestration) that is infeasible within a one-semester timeline.

- Monolithic Architecture without clear separation (Rejected): Would lead to tight coupling between modules, making it difficult to test components independently and violating the maintainability NFR (>80% test coverage).

UML diagrams (component, class, sequence) are used throughout this document as the primary design notation, consistent with the course guidelines

3. Architectural and Component-Level Design

This section presents the system architecture and provides detailed specifications for each major component of myClinic.

3.1 System Structure

myClinic follows a three-tier layered architecture: Presentation Layer (frontend), Application Layer (Django backend with REST API), and Data Layer (PostgreSQL database). External services (AI API, SMTP) communicate with the Application Layer only.

The system is divided into six major components, each corresponding to a functional requirement group:

- C1: Authentication & Authorization Module
- C2: Appointment & Dynamic Pricing Module
- C3: AI Integration Module (Test Analysis & Pricing Engine)
- C4: Medical Records & Prescription Module
- C5: Messaging Module
- C6: Analytics & Leave Management Module

3.1.1 Architecture Diagram

The UML Component Diagram below illustrates the system architecture and the relationships between components:

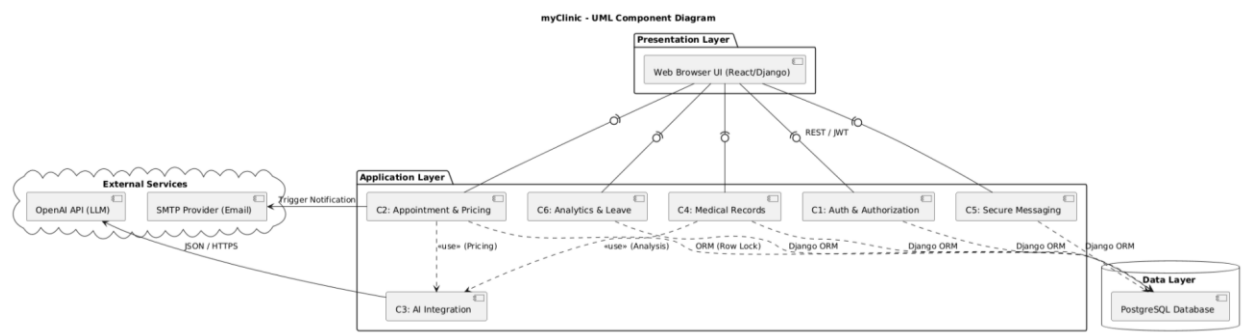


Figure 3.1: myClinic System Architecture, UML Component Diagram

The architecture enforces the following key design principle: no component communicates directly with the database except through the Django ORM layer. All external service calls are routed exclusively through C3 (AI Integration) or the Django email backend.

3.2 C1: Authentication & Authorization Module

3.2.1 Processing Narrative (PSPEC)

C1 is responsible for all user identity verification and access control. It handles user registration, login, password hashing, JWT token issuance, and RBAC enforcement across all API endpoints. Every request to any other module must first pass through C1's authorization middleware, which validates the JWT and confirms that the user's role permits the requested action.

3.2.2 Interface Description

Attribute	Description
Input	POST /api/auth/login/ with { email, password }; POST /api/auth/register/ with { name, email, password } (role excluded; backend auto-assigns Patient role). POST /api/users/ with { name, email, password, role } (Manager-only endpoint for Doctor/Manager provisioning)
Output	{ access_token, refresh_token, role } on success; { error: 'Invalid credentials' } on failure
Dependencies	Django ORM (User model), djangorestframework-simplejwt
Consumed by	All other modules (via middleware)

3.2.3 Processing Detail

3.2.3.1 Design Class: User (Base) extended by Patient, Doctor, and Manager subclasses using Django's AbstractUser.

3.2.3.2 Restrictions: Passwords must be at least 8 characters; roles cannot be self-assigned by users after registration.

3.2.3.3 Performance: Token validation must complete in under 50ms (in-memory JWT verification).

3.2.3.4 Design Constraints: Role assignment is controlled by an admin or manager; patients self-register but are assigned the 'Patient' role automatically.

3.2.3.5 Key Operations:

- `authenticate(email, password)`: Verifies credentials using bcrypt hash comparison; returns JWT on success.
- `authorize(token, required_role)`: Validates JWT signature and expiry; checks role claim against `required_role`.

- `refresh_token(refresh)`: Issues a new access token using a valid refresh token.
- `logout(token)`: Adds the current access token to the TokenBlacklist table with its expiry timestamp, immediately invalidating it before natural expiry.

3.2.4 Dynamic Behavior: Interaction Diagram

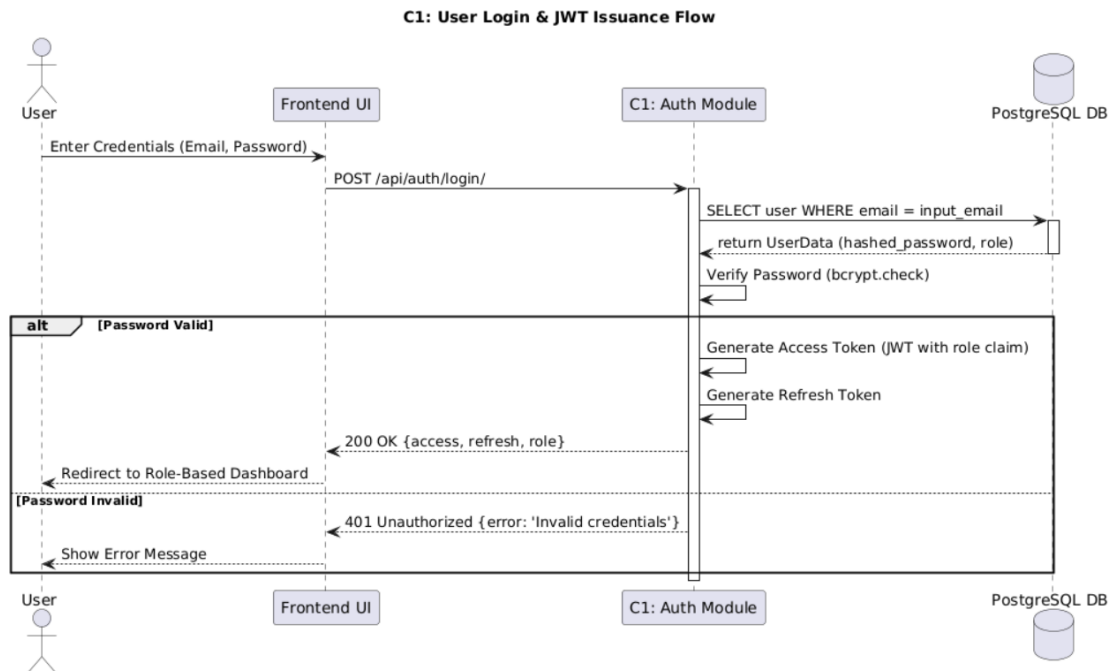


Figure 3.2: Sequence Diagram, C1: User Login and Token Issuance

3.3 C2: Appointment & Dynamic Pricing Module

3.3.1 Processing Narrative (PSPEC)

C2 manages the complete appointment lifecycle: creation, modification, cancellation, and status management. It also coordinates with C3 (AI Integration Module) to fetch a dynamically calculated fee before confirming a booking. The module enforces business rules such as preventing double-bookings, blocking past-date scheduling, and applying database-level locking to handle concurrent booking requests.

3.3.2 Interface Description

Attribute	Description
Inputs	GET /api/appointments/available/?doctor_id=X&date=Y to fetch slots; POST /api/appointments/ with { doctor_id, date_time }; DELETE /api/appointments/{id}/

Attribute	Description
Outputs	Appointment object with status and calculatedFee; confirmation email trigger
Dependencies	C1 (auth), C3 (pricing), C6 (leave schedule), Email Backend, PostgreSQL
Exposed to	Patient role (booking), Doctor role (view/manage schedule), Manager role (overview)

3.3.3 Processing Detail

3.3.3.1 Design Classes: Appointment { appointmentID, patientID, doctorID, dateTime, status, calculatedFee, patient_rating (Integer 1-5, nullable) }; AppointmentService (business logic); AppointmentSerializer (DRF)

3.3.3.2 Restrictions: A patient cannot hold overlapping appointments; a doctor cannot be booked during approved leave periods.

3.3.3.3 Performance: Availability check query must complete within 500ms using indexed queries on (doctorID, dateTime).

3.3.3.4 Design Constraints: Uses SELECT FOR UPDATE (PostgreSQL row-level lock) to prevent race conditions on simultaneous bookings.

3.3.3.5 Key Operations:

- get_available_slots(doctor_id, date): Queries Appointment and LeaveRequest tables; returns free slots.
- create_appointment(patient_id, doctor_id, date_time): Acquires row lock, calls C3.calculate_fee(), saves record, then triggers email.
- cancel_appointment(appointment_id): Validates ownership and status; updates status to 'Cancelled'; releases slot.

3.3.4 Dynamic Behavior: Interaction Diagram

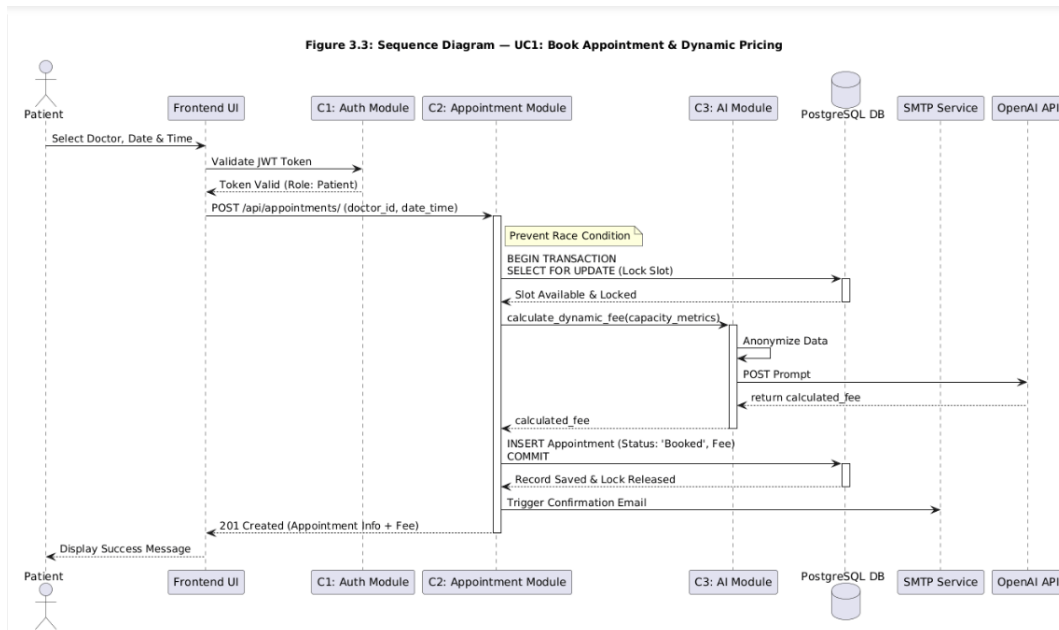


Figure 3.3: Sequence Diagram, C2: Book Appointment and Dynamic Pricing

3.4 C3: AI Integration Module

3.4.1 Processing Narrative (PSPEC)

C3 serves as the system's sole interface to the external AI (LLM) API. It is responsible for two distinct AI functions: (1) dynamic appointment fee calculation based on clinic capacity metrics, and (2) medical test analysis with diagnostic suggestions. C3 handles PII anonymization before sending data externally, manages API timeouts, and implements graceful degradation to ensure core system operations continue even when the AI service is unavailable.

3.4.2 Interface Description

Attribute	Description
Inputs (Pricing)	{ doctor_id, date_time, current_booked_slots, total_slots, base_fee, avg_patient_rating } from C2. avg_patient_rating is the mean of patient_rating scores from all COMPLETED appointments for that doctor; null for new doctors (falls back to occupancy-only pricing).
Inputs (Analysis)	{ anonymized_test_data, test_type } from C4
Outputs (Pricing)	{ calculated_fee: float } or fallback to base_fee on timeout. Fee reflects both occupancy rate and doctor quality signal (avg_patient_rating).
Outputs (Analysis)	{ anomalies: [...], suggestions: string, disclaimer: string } or error flag
External Dependency	OpenAI API (RESTful, JSON), requires API_KEY env variable

Attribute	Description
Timeout Policy	Pricing requests: 3-second hard timeout; on failure, falls back to base_fee with a warning flag. Analysis requests: 8-second hard timeout; on failure, returns a degraded response with a Service Unavailable warning while still allowing the doctor to proceed manually.

3.4.3 Processing Detail

3.4.3.1 Design Classes: AIService (singleton service class); PricingPromptBuilder; AnalysisPromptBuilder; AIResponseParser

3.4.3.2 Restrictions: Patient PII (name, ID, contact) must be stripped before sending any data to the external API. The module must never store raw AI responses containing patient data. Strict prompt engineering techniques (e.g., input escaping, system/user context boundaries, role-based prompt separation) must be applied to mitigate Prompt Injection attacks, particularly when doctor-authored free-text notes are included in analysis requests.

3.4.3.3 Performance: Pricing requests must complete within 3.0 seconds; analysis requests within 8.0 seconds. Uses async HTTP calls (Python httpx) to avoid blocking the main thread.

3.4.3.4 Design Constraints: API key stored as environment variable (not in codebase). Prompt templates versioned and testable.

3.4.3.5 Key Operations:

- `calculate_dynamic_fee(capacity_metrics)`: Constructs pricing prompt including occupancy rate and avg_patient_rating, calls LLM API, parses fee, and returns float.
- `analyze_test_results(anonymized_data)`: Constructs analysis prompt, calls LLM API, parses anomalies and suggestions, then returns a structured response.
- `_handle_api_failure(function_type)`: Returns { fallback: true, message: 'Service Unavailable' } for graceful degradation.

3.4.4 Dynamic Behavior: Interaction Diagram

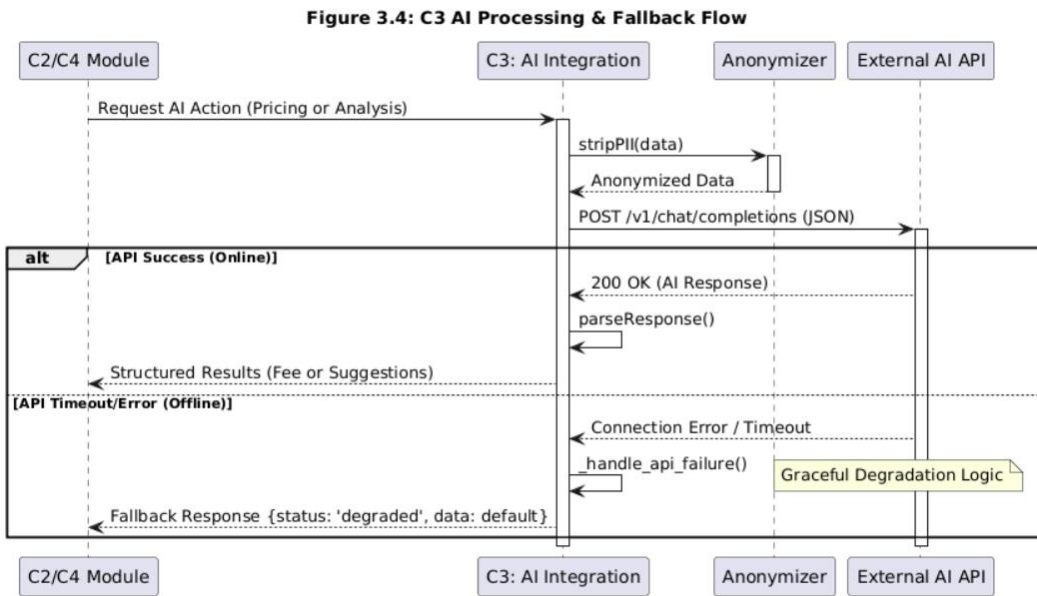


Figure 3.4: Sequence Diagram, C3: AI-Assisted Medical Test Analysis

3.5 C4: Medical Records & Prescription Module

3.5.1 Processing Narrative (PSPEC)

C4 manages all persistent medical data for patients, including medical history, AI-assisted test analyses (post-doctor approval), and digital prescriptions. It enforces the constraint that prescriptions can only be created by Doctors and must be linked to both a valid patient and appointment record. All sensitive data stored by this module is encrypted at rest (AES-256).

3.5.2 Interface Description

Attribute	Description
Inputs (Records)	POST /api/records/ with { patient_id, analysis_data, doctor_notes } (Doctor role only)
Inputs (Prescriptions)	POST /api/prescriptions/ with { patient_id, appointment_id, drugs: [{name, dosage, instructions}] }
Outputs	Medical record and prescription objects; visible on Patient Dashboard (read-only for patients)
Dependencies	C1 (auth/RBAC), C3 (AI analysis results), PostgreSQL (with AES-256 at-rest encryption)

3.5.3 Processing Detail

3.5.3.1 Design Classes: MedicalRecord { recordID, patientID, doctorID, date, analysisData, doctorNotes, aiSuggestions, doctorApproved }; Prescription { prescriptionID, patientID, appointmentID, doctorID, drugs, createdAt }

3.5.3.2 Restrictions: Prescriptions cannot be saved if required fields (drug name, dosage) are blank. AI suggestions are saved only after explicit doctor approval (boolean flag).

3.5.3.3 Performance: Medical history retrieval paginated (max 20 records/page) to meet the 2-second response NFR.

3.5.3.4 Design Constraints: Data retention policy requires records to be stored for a minimum of 10 years per health regulations. To reconcile this with the KVKK/GDPR Right to be Forgotten, the system applies a soft delete strategy: when a patient requests account deletion, their Patient record is deactivated (is_active=False) and all PII fields (fullName, dateOfBirth, phoneNumber) are set to null. Medical records and appointments are retained with anonymized references to comply with health regulations while respecting privacy rights.

3.5.3.5 Key Operations:

- create_prescription(doctor_id, patient_id, appointment_id, drugs): Validates required fields, links to appointment, then saves record.
- approve_analysis(record_id, doctor_id): Flips doctorApproved flag; makes the AI analysis part of the official record.
- get_patient_history(patient_id): Returns paginated list of all medical records and prescriptions for a patient.

3.5.4 Dynamic Behavior: Interaction Diagram

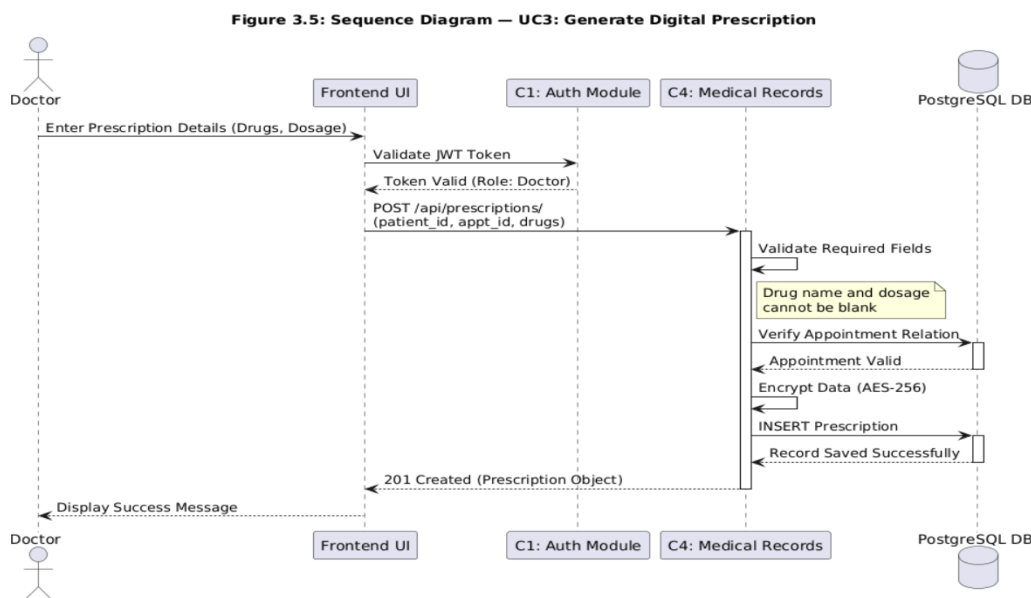


Figure 3.5: Sequence Diagram, C4: Generate Digital Prescription

3.6 C5: Secure Messaging Module

3.6.1 Processing Narrative (PSPEC)

C5 provides an asynchronous, secure messaging channel with server-side encryption at rest between patients and their doctors. It enforces the constraint that a patient may only initiate a conversation with a doctor they have previously had an appointment with. All messages are encrypted before database storage, and unread messages trigger a visual notification badge on the doctor's dashboard.

3.6.2 Interface Description

Attribute	Description
Inputs	POST /api/messages/ with { doctor_id, message_text } (Patient role only); GET /api/messages/{conversation_id}/ (Patient or Doctor)
Outputs	Encrypted message stored in DB; unread badge count on Doctor dashboard
Validation	Message must be 1–500 characters; patient must have at least one Completed appointment with the doctor
Dependencies	C1 (auth), C2 (appointment history lookup), PostgreSQL

3.6.3 Processing Detail

3.6.3.1 Design Classes: Message { messageID, senderID, receiverID, encryptedContent, sentAt, isRead }; MessageService

3.6.3.2 Restrictions: No messages allowed between unrelated patient-doctor pairs. Message content truncated server-side at 500 characters as a secondary validation.

3.6.3.3 Performance: Message delivery (save to DB + trigger notification) must complete within 1.0 second.

3.6.3.4 Design Constraints: Message content encrypted using AES-256 symmetric key before storage. The key is managed via Django's SECRET_KEY mechanism.

3.6.3.5 Key Operations:

- send_message(patient_id, doctor_id, text): Validates appointment relationship, encrypts text, saves to DB, then increments unread count.
- get_conversation(patient_id, doctor_id): Retrieves and decrypts messages for the conversation thread.
- mark_as_read(message_ids, user_id): Updates isRead flag for messages in the conversation.

3.6.4 Dynamic Behavior: Interaction Diagram

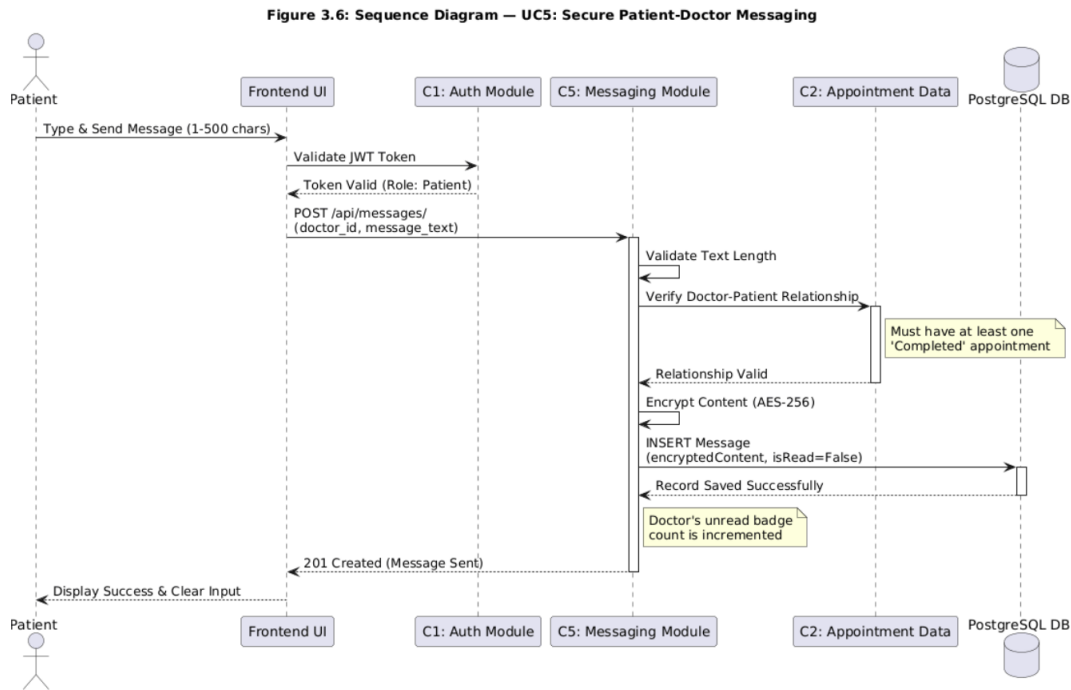


Figure 3.6: Sequence Diagram, C5: Secure Patient-Doctor Messaging

3.7 C6: Analytics & Leave Management Module

3.7.1 Processing Narrative (PSPEC)

C6 handles two related administrative concerns. First, it processes doctor annual and sick leave requests: validating dates, checking for appointment conflicts, and notifying the Manager. Second, it powers the Manager's analytics dashboard by aggregating appointment, revenue, and doctor performance data from the database for a selected date range. Both sub-functions are restricted to specific roles (Doctor for leave, Manager for analytics).

3.7.2 Interface Description

Attribute	Description
Inputs (Leave)	POST /api/leave-requests/ with { start_date, end_date, reason, leave_type: 'Annual'/'Sick' } (Doctor role only); PATCH /api/leave-requests/{id}/ with { status: 'Approved'/'Rejected' } (Manager role only)
Inputs (Analytics)	GET /api/analytics/?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD (Manager role only); returns all KPI metrics for the selected period in a single response

Attribute	Description
Outputs (Leave)	LeaveRequest object with 'Pending' status; notification on Manager dashboard; schedule block on approval
Outputs (Analytics)	Single JSON object containing: period { start_date, end_date }, total_appointments (integer), total_revenue (float, sum of calculatedFee for all COMPLETED appointments; payment assumed collected in person), vacancy_rate (float, 0-1), avg_satisfaction_score (float, 1-5), doctor_performance [{ doctor_id, name, appointment_count, avg_rating (float, 1-5) }]. avg_rating is calculated from patient_rating scores on COMPLETED appointments and is also fed into the C3 dynamic pricing engine.
Dependencies	C1 (auth/RBAC), C2 (appointment data), PostgreSQL

3.7.3 Processing Detail

3.7.3.1 Design Classes: LeaveRequest { leaveID, doctorID, startDate, endDate, reason, status, managerID }; AnalyticsService

3.7.3.2 Restrictions: Leave cannot be requested for past dates. If appointments exist during requested leave dates, the system returns a warning with conflicting appointment IDs.

3.7.3.3 Performance: Analytics queries must complete within 2.0 seconds; complex aggregations use pre-computed or indexed queries.

3.7.3.4 Design Constraints: Analytics endpoint is read-only (no mutations). When a leave request is approved, its status is updated to Approved in the LeaveRequest table. No additional blocking mechanism is required: C2 module's get_available_slots() already queries the LeaveRequest table and excludes approved leave dates from available slots automatically.

3.7.3.5 Key Operations:

- submit_leave_request(doctor_id, start_date, end_date, reason): Checks for conflicts, creates LeaveRequest with status Pending, then notifies Manager.
- approve_reject_leave(leave_id, manager_id, decision): Updates LeaveRequest status to Approved or Rejected. No further action required; C2 module automatically excludes approved leave dates when computing available slots.
- get_analytics(start_date, end_date): Runs a single aggregated DB query and returns a structured dict with all KPI metrics covering appointments, revenue, vacancy rate, satisfaction scores, and per-doctor performance.

3.7.4 Dynamic Behavior: Interaction Diagrams

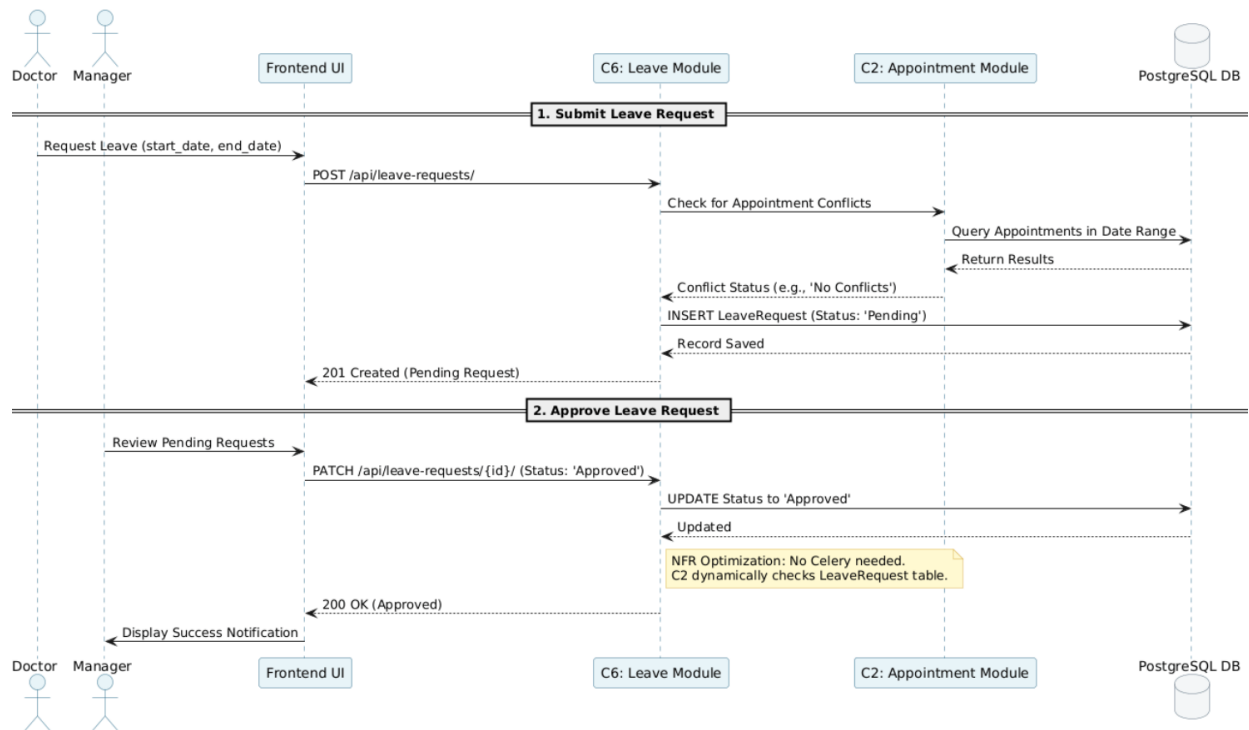


Figure 3.7a: Sequence Diagram, C6: Doctor Leave Management

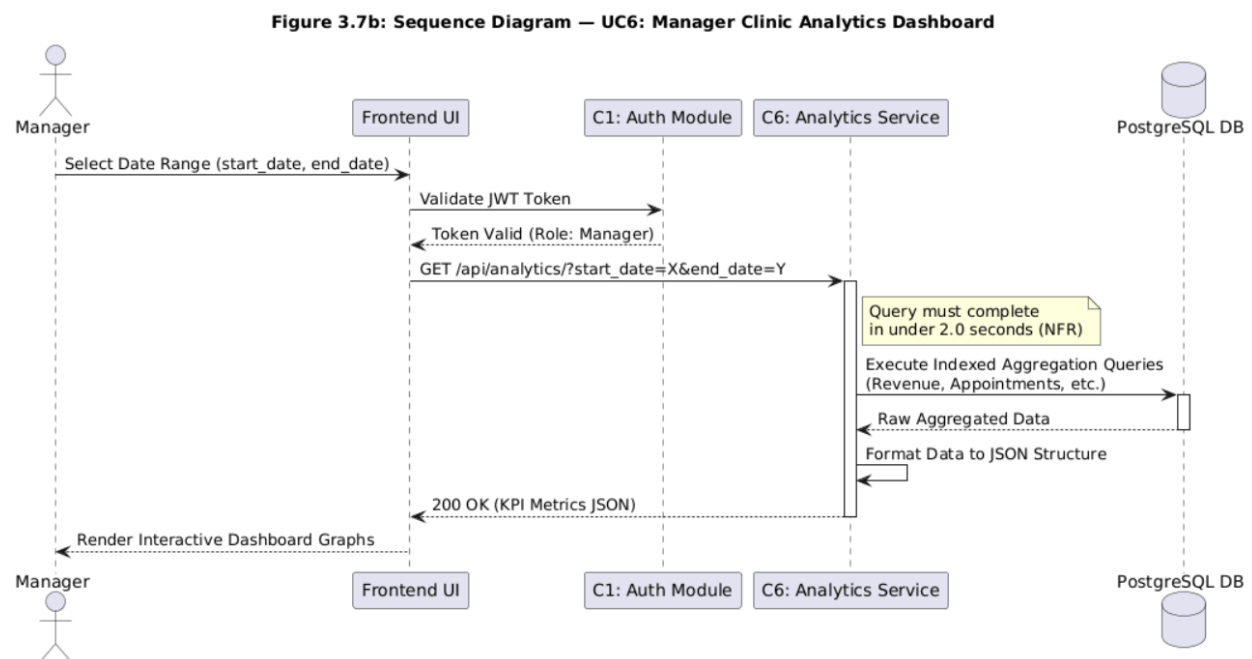


Figure 3.7b: Sequence Diagram, C6: Manager Clinic Analytics Dashboard

3.9 Class Diagram

The following UML Class Diagram presents the complete object model for myClinic. All class attributes and methods are consistent with the SRS class definitions and the Data Model in Section 4. Enumerations (RoleEnum, ApptStatusEnum, LeaveStatusEnum) are shown separately. The diagram reflects the security decision to exclude the role field from the self-registration endpoint (Decision 5).

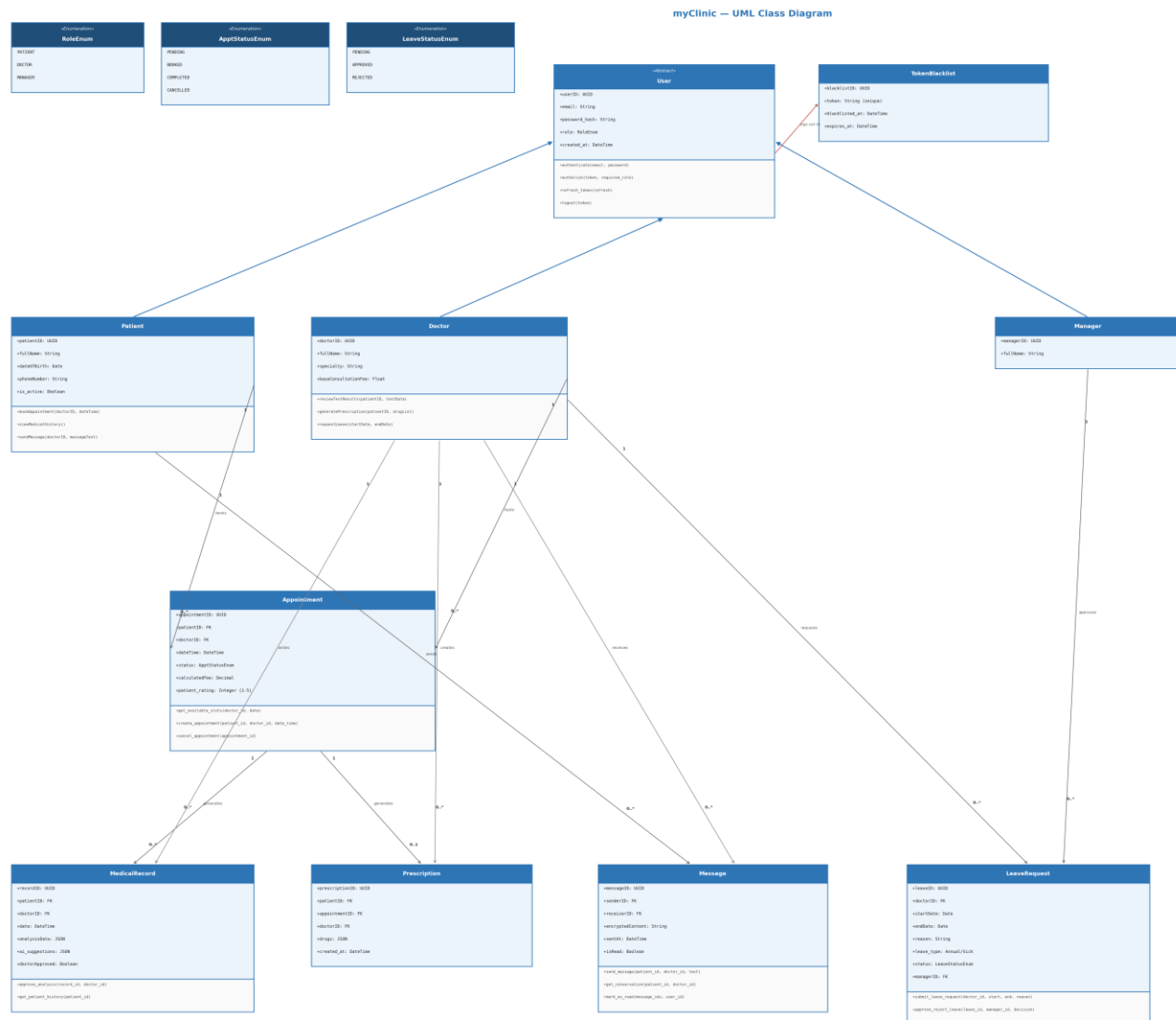


Figure 3.8: UML Class Diagram, myClinic System

4. Data Model

The following table describes the core database entities, their primary attributes, and their relationships. The database uses PostgreSQL with Django ORM. All foreign key constraints are enforced at the database level.

Entity	Key Attributes	Relationships
User (abstract)	userID (PK), email, password_hash, role	Base for Patient, Doctor, Manager
Patient	patientID (PK, FK→User), fullName, dateOfBirth, phoneNumber, is_active (Boolean, default True). On deletion request: is_active=False and PII fields set to null (soft delete per KVKK/GDPR)	Has many Appointments, Messages, MedicalRecords
Doctor	doctorID (PK, FK→User), specialty, baseConsultationFee	Has many Appointments, Prescriptions, LeaveRequests
Manager	managerID (PK, FK→User)	Approves LeaveRequests; views Analytics
Appointment	appointmentID (PK), patientID (FK), doctorID (FK), dateTime, status, calculatedFee, patient_rating (Integer, 1-5)	Linked to Patient, Doctor, Prescription
MedicalRecord	recordID (PK), patientID (FK), doctorID (FK), date, analysisData, doctorApproved	Linked to Patient, Doctor
Prescription	prescriptionID (PK), patientID (FK), appointmentID (FK), doctorID (FK), drugs (JSON)	Linked to Appointment
Message	messageID (PK), senderID (FK), receiverID (FK), encryptedContent, sentAt, isRead	Between Patient and Doctor
LeaveRequest	leaveID (PK), doctorID (FK), startDate, endDate, reason, status, managerID (FK)	Linked to Doctor and Manager
TokenBlacklist	blacklistID (PK), token (String, unique), blacklisted_at (DateTime), expires_at (DateTime)	Used by C1 logout(); checked by Auth Middleware on every protected request

5. Restrictions, Limitations, and Constraints

The following design issues have a significant impact on the implementation and future evolution of myClinic:

- **External AI Dependency:** The system's dynamic pricing and test analysis features are entirely dependent on a third-party LLM API. Any breaking changes to the API (versioning, rate limits, cost increases) require updates to C3 prompt templates and parsing logic. This is the most significant source of technical debt in the current design.
- **No Real-Time Payment Processing:** The system calculates and displays fees but does not process payments. Any future commercialization requires integration of a payment gateway (e.g., Stripe, Iyzico), which would require significant new components and security auditing.
- **Single-Clinic Deployment:** The current database schema and authentication model assume a single clinic. Multi-tenancy (supporting multiple clinics) would require a major schema redesign (e.g., adding a Clinic entity and partitioning all data by clinic).
- **Message Encryption Key Management:** Messages are encrypted using Django's SECRET_KEY. Rotating this key would render existing encrypted messages unreadable. A proper key management system (e.g., AWS KMS) should be adopted in a production environment.
- **Scope of AI Analysis:** The AI module is limited to text-based lab results and radiology reports input manually by the doctor. It cannot process binary image formats (DICOM) or interface directly with lab hardware, limiting automation potential.
- **Session Security:** Sessions expire after 15 minutes of inactivity (per NFR). This may frustrate doctors in the middle of writing a lengthy prescription. A configurable timeout with a soft-warning dialog should be considered in future iterations.

6. Conclusion

This Software Design Specification defines a complete, coherent architectural blueprint for the myClinic web application. The system is structured into six well-defined, loosely coupled components that collectively address all functional requirements identified in the SRS while satisfying the non-functional requirements for performance, security, maintainability, and portability.

Key design decisions documented in this specification include:

- Adopting a layered MVC architecture (over microservices) to balance clean separation of concerns with feasibility within a one-semester timeline.
- Isolating all external AI communication within a single module (C3) to contain the risk of third-party API changes and enable graceful degradation.
- Using PostgreSQL row-level locking (SELECT FOR UPDATE) in C2 to safely handle concurrent booking requests without race conditions.
- Encrypting all sensitive message content and medical data at rest using AES-256, with RBAC enforced at every API endpoint.

Appendix A: Decision Log

This log documents the key engineering and design decisions made during the Design Specification phase, as required by the course guidelines.

Decision 1: MVC (Layered) Architecture over Microservices

Field	Details
Decision	Adopt a monolithic layered MVC architecture using Django REST Framework.
Alternatives Considered	Microservices architecture (separate services for Auth, Appointments, AI, etc.).
Rationale	Microservices require significant DevOps overhead (Docker orchestration, service discovery, inter-service auth) that is infeasible within one semester. The layered MVC provides sufficient separation of concerns for the project scale.
Trade-offs	Sacrifices horizontal scalability of individual components for faster development and simpler testing.
Risks Technical Debt	/ If the system needs to scale to multiple clinics or high traffic, refactoring into microservices would require significant architectural changes.

Decision 2: Isolating AI in a Single Integration Module (C3)

Field	Details
Decision	All interactions with the external LLM API are routed through a single, dedicated AIService module (C3).
Alternatives Considered	Calling the AI API directly from C2 (appointments) and C4 (medical records) where it is needed.
Rationale	Centralizing AI communication in one module makes it easy to swap providers, update prompt templates, manage API keys, and implement consistent error handling and graceful degradation without touching business logic in other components.
Trade-offs	Adds an indirect communication hop between components, but the performance impact is negligible.
Risks Technical Debt	/ C3 becomes a potential single point of failure for AI-dependent features; mitigated by the graceful degradation design.

Decision 3: PostgreSQL Row-Level Locking for Concurrent Bookings

Field	Details
Decision	Use SELECT FOR UPDATE in C2 to serialize concurrent appointment booking requests at the database level.
Alternatives Considered	(1) Application-level locking using Django's cache-based lock; (2) Optimistic locking with version fields.
Rationale	Database-level locking is the most reliable solution for a relational DB, as it prevents race conditions even across multiple application server instances. Optimistic locking would require complex retry logic in the API layer.
Trade-offs	Slightly reduces booking throughput under extreme concurrency, but well within acceptable bounds for a single-clinic deployment.
Risks / Technical Debt	Long-running transactions holding locks could cause timeouts; mitigated by keeping the booking transaction short.

Decision 4: JWT for Stateless Authentication (over Django Sessions)

Field	Details
Decision	Use JWT tokens (via django-rest-framework-simplejwt) instead of Django's default session-based authentication.
Alternatives Considered	Django's built-in session authentication stored in the database.
Rationale	JWT enables stateless authentication, which is essential for a REST API consumed by a separate frontend. It scales horizontally without requiring a shared session store.
Trade-offs	JWT tokens cannot be invalidated server-side before expiry (unless a token blacklist is maintained), slightly complicating logout and session revocation.
Risks / Technical Debt	Token blacklisting for logout requires an additional database table or Redis store; this must be implemented to prevent security issues.

Decision 5: Patient Rating as a Dynamic Pricing Input

Field	Details
Decision	Patient ratings (1-5) collected after completed appointments are included as avg_patient_rating in the C3 dynamic pricing engine, alongside occupancy metrics. The C6 analytics dashboard also exposes avg_rating per doctor for Manager review.
Alternatives Considered	(1) Pure occupancy-based pricing with no quality signal. (2) Fixed pricing with no dynamic adjustment.
Rationale	Occupancy alone does not reflect doctor quality. A highly booked but poorly rated doctor should not command the same premium as a highly booked and highly rated one. Including avg_patient_rating produces a fairer pricing signal. Implementation cost is minimal: one additional field in the API call.
Trade-offs	New doctors with no ratings yet will have a null avg_patient_rating; the system falls back to occupancy-only pricing in this case.
Risks Technical Debt	Patients may submit unfair ratings. A minimum rating count threshold (e.g., at least 5 ratings) before avg_rating influences pricing should be considered in future iterations.

Appendix B: AI Tools Usage Declaration

In accordance with the course guidelines, this appendix declares the use of AI tools during the preparation of this Design Specification Document.

- **Phases Used:** Structuring the DSD sections according to the course template, brainstorming component decomposition, and drafting initial interface descriptions.

Critical Evaluations and Corrections Made:

- **Component Decomposition:** The AI initially suggested a flat structure with one large 'AppointmentService' handling both booking logic and analytics. We rejected this and applied the Single Responsibility Principle to separate concerns into six distinct components, ensuring each component maps to a clear functional requirement group.
- **AI Module Placement:** The AI suggested placing the LLM API calls directly inside the appointment booking function. We redesigned this to isolate all AI communication in a dedicated C3 module, enabling better error handling, testability, and future provider substitution.
- **Database Encryption Advice:** The AI initially recommended field-level encryption for all database columns. After evaluation, we determined that encryption at the storage level (AES-256 at rest) combined with TLS for transit provides the required security posture without the significant performance overhead of per-field encryption for all data types.

Our group takes full responsibility for the correctness, completeness, and coherence of this document. All architectural decisions have been actively reviewed, discussed, and approved by all team members.